Python Topics to Revise for TCS NQT

For **TCS NQT (Python)** the questions usually focus on **core language concepts + commonly used built-ins + basic problem solving**. Based on previous NQT patterns, these are the **most important Python topics to revise**.

Focus on these first:

1. **Lambda + map + filter**
2. **Counter**
3. **heapq**
4. **Sorting with lambda**
5. **List comprehension**
6. **Slicing**
7. **String methods**
8. **Dictionary operations**
9. **Exception handling**
10. **zip / enumerate**
11. **itertools basics**

1. Important Built-in Functions

These are frequently used in coding questions.

- len()
- sum()
- min(), max()
- sorted()
- reversed()
- enumerate()
- zip()
- abs()
- round()
- pow()
- all(), any()

Example:

```
nums = [3,1,5,2]
print(sorted(nums))   # [1,2,3,5]
print(sum(nums))      # 11
```

2. Functional Programming

Very commonly asked in **MCQ + output prediction**.

Lambda

Anonymous function.

```
square = lambda x: x*x
print(square(5))   # 25
```

map()

Applies a function to all elements.

```
nums = [1,2,3]
result = list(map(lambda x: x*2, nums))
print(result)   # [2,4,6]
```

filter()

Filters elements based on condition.

```
nums = [1,2,3,4]
result = list(filter(lambda x: x%2==0, nums))
print(result)   # [2,4]
```

reduce() (from functools)

```
from functools import reduce

nums = [1,2,3,4]
result = reduce(lambda a,b: a+b, nums)
print(result)   # 10
```

3. collections Module (Very Important)

Counter

Counts frequency.

```
from collections import Counter

arr = [1,1,2,3,3,3]
c = Counter(arr)
print(c)
```

Output

Counter({3:3, 1:2, 2:1})

Common Questions:

- Most frequent element
- Character frequency in string

defaultdict

```
from collections import defaultdict

d = defaultdict(int)
d["a"] += 1
print(d)
```

4. heapq (Priority Queue)

Used for **top k problems, min/max elements**.

```
import heapq

nums = [5,1,8,3]

heapq.heapify(nums)
print(heapq.heappop(nums))   # 1
```

Important functions

- heapify()
- heappush()
- heappop()
- nlargest()
- nsmallest()

Example:

```
heapq.nlargest(2, [4,1,7,3,9])
```

Output

[9,7]

5. String Methods

Very frequently asked.

Important ones:

split()

join()

replace()

strip()

upper()

lower()

count()

find()

startswith()

endswith()

Example

```
s = "hello world"
print(s.split())
print("-".join(["a","b","c"]))
```

6. List Methods

append()

extend()

insert()

remove()

pop()

index()

count()

sort()

reverse()

Example

```
arr = [3,1,2]
arr.sort()
print(arr)
```

7. Slicing (Very Important)

Slicing is used to **extract portions of lists, strings, or tuples**.

Syntax

```
sequence[start : end : step]
```

Examples

Reverse a string

```
s = "python"
print(s[::-1])
```

Output

nohtyp

Subarray from a list

```
arr = [10,20,30,40,50]
print(arr[1:4])
```

Output

[20, 30, 40]

Skipping elements

```
arr = [1,2,3,4,5,6]
print(arr[::2])
```

Output

[1,3,5]

8. List / Dictionary Comprehension

Very common.

List comprehension

```
squares = [x*x for x in range(5)]
```

Dictionary comprehension

```
d = {x:x*x for x in range(5)}
```

9. Itertools (Occasionally Asked)

permutations

combinations

product

Example

```
from itertools import permutations

print(list(permutations([1,2,3],2)))
```

10. Python Data Structures

Must revise:

List

Mutable

Tuple

Immutable

Set

Unique elements

Dictionary

Key-value pairs

Example

```
s = {1,2,2,3}
print(s)   # {1,2,3}
```

11. Exception Handling

Basic syntax.

```
try:
   x = 10/0
except ZeroDivisionError:
   print("Error")
```

12. Python Sorting Tricks

Very important.

Sort with lambda

```
arr = [(1,3),(2,1),(5,2)]

arr.sort(key=lambda x: x[1])
print(arr)
```

13. Python Interview Tricks (Very Common MCQs)

Swap without temp

```
a,b = b,a
```

Reverse string

```
s[::-1]
```

Remove duplicates

```
list(set(arr))
```

14. Basic Algorithms

Revise Python implementation of:

- Binary Search
- Frequency Count
- Two Pointer
- Prefix Sum
- Sorting

15. Time Complexity Basics

Know complexity of common operations.

Operation	Complexity
list append	O(1)
list insert	O(n)
dictionary lookup	O(1)
set lookup	O(1)
sorting	O(n log n)